

Een if-statement dat je code niet kan schrijven

TypeSafe **Jev**: een "System One"-model dat getypeerde beslissingen geeft, geen tekst

20 minuten · slides + live demo · alles in één marimo-notebook



LIVE API (uit = offline, recordings/)

opnieuw uitvoeren

De hook: LLM-output parsen is fragiel

```
prompt = f"""Classify this ticket as billing, technical or sales.  
Answer with ONLY the label.
```

```
{ticket}"""
```

```
label = llm(prompt).strip().lower()
```

```
if label == "billing":          # "Billing.", "billing (probably)",  
    route_to_billing()        # "I'd say billing", ...
```

```
else:  
    ...                        # en hoe zeker was het model eigenlijk?
```

- Tekst → string-parsing → **retries, regexes, JSON-mode, schema-validators**
- Je krijgt een antwoord, maar **geen kans** dat het klopt
- 1-3 s en een paar cent per beslissing

Wat we eigenlijk willen: if $P(\text{billing}) > 0.8$:

System 1 vs System 2 (Kahneman)

System 1 · snel, intuïtief

- Herkennen, inschatten, kiezen
- Milliseconden, geen redenering
- *"Dit is een billing-ticket, en dringend"*

→ **Jev** (System One-model)

System 2 · traag, bewust

- Redeneren, plannen, schrijven
- Seconden, stap voor stap
- *"Laat me dit ticket eens goed lezen en een antwoord opstellen"*

→ **LLM** (GPT, Claude, ...)

Beide heb je nodig. De vraag is: **welke beslissingen laat je System 2 nemen die eigenlijk System 1-werk zijn?**

Jev vs LLM

LLM		Jev
Input	prompt (tekst)	state (string/JSON) + getypeeerde vragen
Output	tekst, hopelijk in het juiste formaat	getypeerd antwoord + kansen , schema-conform gegarandeerd
Redenering	chain-of-thought, uitleg	geen (er komt geen reasoning terug)
Latency	1-10 s	70-500 ms (vendor-claim)
Prijs	€ per M tokens in + uit	\$0.042 / M input-tokens, output gratis (vendor)
Goed in	genereren, redeneren	kiezen, scoren, ja/nee — gecalibreerd

Meerdere vragen over dezelfde state gaan in **één call** en worden parallel beantwoord.

Drie primitives + confidence

Confidence = hoe geconcentreerd de verdeling is. Niet: hoe *juist* het antwoord is.

```
QUESTIONS = {
    "team": Choice(
        instructions="Which team should own this ticket?",
        criteria=TEAMS,          # label -> description (or None)
    ),
    "urgency": Score(
        instructions="How urgent is this ticket for the customer?",
        criteria=URGENCY_LEVELS, # ordered rubric: index = score
    ),
    "refund": Noul(
        instructions="Does the customer ask for money back?",
        criteria=REFUND,        # optional: describe yes / no
    ),
}

URGENCY_LEVELS = [
    "No time pressure; can wait a week.",    # score 0
    "Should be handled within a few days.",  # score 1
    "Blocking the customer; needs a same-day response.", # score 2
]
```

Choice

kies één optie → `.choice`, `.probabilities` (som ≈ 1),
`.confidence`

Score

niveau op een geordende rubriek → `.score` (gewogen),
`.probabilities` per niveau, `.confidence`

Noul

ja/nee → `.noul` = $P(\text{ja})$. Geen aparte confidence: 0.5 betekent *weet het niet*

Demo 1 · Smart if

OFFLINE - opgenomen API-antwoord

Hi, I upgraded to the Pro plan on Monday but my invoice shows I was charged twice (EUR 49 and again EUR 49). I need one of those charges reversed. On top of that the export to CSV has been failing all week with a 500 error and my quarterly report is due tomorrow morning, so please look at this today.

```
def smart_if(client: TypeSafeClient, ticket: str) -> SmartIfResult:
    response, latency_ms = timed_system_one(
        client, state={"ticket": ticket}, questions=QUESTIONS
    )
    team = response.choices["team"]          # .choice, .confidence, .probabilities
    urgency = response.scores["urgency"]     # .score, .probabilities per level
    refund = response.nouls["refund"]        # .noul = P(yes)
    return SmartIfResult(
        ticket=ticket,
        team=view(team),
        urgency=view(urgency),
        refund=view(refund),
        meta=CallMeta.of(response, latency_ms),
    )
```

1006 ms · één call · model `jev-1.13.0` · 567 input-tokens

billing

team · conf. 0.59

2/2

urgency · Blocking the customer

ja

refund? · P(ja) 0.98

team	P
billing	0.70
technical	0.25
other	0.05
sales	0.00

	urgency	P
	0 · No time pressure	0.00
1 · Should be handled within a few days.		0.00
2 · Blocking the customer		1.00

Een if-statement dat je code niet kan schrijven

TypeSafe **Jev**: een "System One"-model dat getypeerde beslissingen geeft, geen tekst

20 minuten · slides + live demo · alles in één marimo-notebook



LIVE API (uit = offline, recordings/)

opnieuw uitvoeren

82%
automatisch (9)
18%
escalatie (2)
3.4 s
totale API-tijd · mediaan 275 ms / p95 512 ms

	ticket	team	confidence (drempel 0.75)	urg.	ms	beslissing
T-101	Your API returns 502 on every request since 09:00 UTC. Our...	technical	1.00	2	722	auto
T-102	I was charged EUR 49 twice this month. Please refund one o...	billing	1.00	1	242	auto
T-103	Do you offer a discount for non-profits if we take the ann...	sales	1.00	0	248	auto
T-104	Where can I download my invoices as PDF? I can't find the ...	billing	0.97	1	275	auto
T-105	The webhook for	technical	0.95	2	285	auto

	ticket	team	confidence (drempe 0.75)	urg.	ms	beslissin
	'invoice.paid' never fires, so our account...					
T-106	How many seats are included in the Team plan, and can we a...	sales	0.98	1	302	aut
T-107	Password reset email never arrives (checked spam). Locked ...	technical	1.00	2	263	aut
T-108	Thanks for the great support last week, no action needed!	other	1.00	0	276	aut
T-109	My subscription renewed but the new features from the upgr...	technical	0.37	1	271	ESCALATI
T-110	Can you tell me what the price would be for 250 users, and...	sales	1.00	1	282	aut

	ticket	team	confidence (drempel 0.75)	urg.	ms	beslissing
	T-111	hello?? still waiting	other	0.54	2 252	ESCALATI

Waar Jev in een agent-loop past

user→Jev: routing→LLM plant→Jev: gate→tool→Jev: eval

Routing · welk model / welke agent / welke tool-set?

`ModelRouterMiddleware` in langchain-typesafe **Gates** · mag deze tool call? is dit klaar? moet een mens kijken?

`AutoModeMiddleware` (1 risk-vraag) of je eigen

middleware **Compaction** · welke berichten zijn nog relevant voor de context? **Evals** · is dit antwoord correct / veilig / on-topic? — per turn, in ms Code houdt de controle; Jev levert *programmable common sense*

Demo 3 · Agent gate — Jev keurt elke tool call vooraf

OFFLINE - opgenomen API-antwoord

```
def wrap_tool_call(self, request: ToolCallRequest, handler: Callable):
    call = request.tool_call
    body = {"state": gate_state(request), "questions": GATE_QUESTIONS}
    response, ms = timed(lambda: self.classifier.invoke(body))
    p = {name: answer.noul for name, answer in response.nouls.items()}
    blocked, reason = decide(p, self.threshold)
    self.decisions.append(
        GateDecision(call["name"], call["args"], p, blocked, reason, round(ms))
    )
    if blocked:
        return blocked_message(call, reason) # LLM sees an error ToolMessage
    return handler(request) # otherwise: run the tool as usual

def decide(p: dict[str, float], t=THRESHOLD) -> Verdict:
    if p["secrets"] >= t:
        return True, "may expose secrets"
    if p["destructive"] >= t and p["production"] >= t:
        return True, "destructive action on production"
    return False, "allowed"
```

#	tool call	destructief?	productie?	sec
1	run_shell(host='app-01', command='df -h')	0.05	0.93	
2	run_shell(host='app-01', command='df -h')	0.03	0.91	
3	run_sql(database='orders- production', query='-- List slowest queries using ...')	0.06	0.97	
4	run_sql(database='orders- production', query='-- Drop the archive table to f...')	0.96	0.97	
5	read_file(host='app-01', path='/etc/app/secrets.env')	0.03	0.50	

#	tool call	destructief?	productief?	seconds
6	run_sql(database='orders-production', query='-- Active queries by duration\n...')	0.04	0.98	
7	run_shell(host='app-01', command='du -sh /var/lib/postgresql/* /...')	0.04	0.90	
8	run_shell(host='app-01', command='du -sh /var/lib/postgresql/* /...')	0.02	0.85	
9	run_shell(host='app-01', command='ls -lh /etc/app true')	0.03	0.88	
10	run_shell(host='app-01', command='ls -lh /etc/app true')	0.02	0.86	
11	run_sql(database='orders-production', query='-- Show table sizes to find bi...')	0.02	0.96	
12	run_sql(database='orders-production', query='SELECT queryid, calls, total_t...')	0.02	0.98	
13	run_shell(host='app-01', command='find /var/lib/postgresql /var/...')	0.02	0.88	
14	run_sql(database='orders-production', query='SELECT schemaname, relname AS ...')	0.02	0.97	
15	run_sql(database='orders-production', query='-- Get size and row count for ...')	0.02	0.98	

#	tool call	destructief?	productie?	sec
16	run_sql(database='orders-production', query='-- Simulated outputs\n-- archiv...')	0.03	0.93	

LLM: openai:gpt-5-mini · 16 tool calls, **4 geblokkeerd** · agent-run 87.7 s · tools zijn gesimuleerd, er wordt niets uitgevoerd

Rapport van de agent: Actions taken 1) Checked disk on app-01 - df -h: /dev/sda1 80G total, 74G used, 6.0G avail (93% used). 2) Investigated slow queries in orders-production - Attempted full pg_stat_statements query but that call was partially blocked by security gating. - Partia...

Dezelfde agent, drie poortwachters

OFFLINE - SYNTHETISCHE PLACEHOLDER, geen echte Jev-output

2.3 s

LLM als rechter: per tool call

0.4 s

Jev als rechter: per tool call

43%

LLM als rechter: aandeel van de run

12%

Jev als rechter: aandeel van de run

6.4 u

LLM als rechter: 10.000 checks, projectie · kost: zet LLM_PRICE_*

1.1 u

Jev als rechter: 10.000 checks, projectie · \$0.07

gate	gate / call	gate-tijd	aandeel run	agent-run	agent zonder gate	tool calls	geblok
geen gate	-	0.0 s	0%	12.6 s	12.6 s	4	
LLM als rechter	2310 ms	9.3 s	43%	21.4 s	12.1 s	4	
Jev als rechter	410 ms	1.7 s	12%	14.2 s	12.5 s	4	

Medianen over 1 run(s) per gate, zelfde taak, zelfde drie vragen, zelfde beleid. Op vier stappen is het verschil één seconde: **de winst zit niet in de wandtijd**. Ze zit in het aandeel van de run dat toezicht is, in de kost per check op schaal, en in kansen met een drempel in plaats van booleans die flippen.

Vendor-claims vs wat ik vanuit België mat

Claims van TypeSafe

- latency **70-500 ms**
- input **\$0.042 / M tokens**, output gratis
- "*193.6× sneller / 444.6× goedkoper*" dan een LLM-workflow, op hun **eigen** workflow-evals — zelf noemen ze dat de bovengrens
- service draait aan de **US West Coast**

Gemeten vanuit België

locatie onbekend

OFFLINE - opgenomen API-antwoord

	demo	n	mediaan	p95	min-max
	demo1 smart if	10	255 ms	280 ms	236-293 ms
	demo2 confidence gate	10	265 ms	293 ms	238-297 ms
	demo3 agent gate	10	409 ms	811 ms	290-1056 ms

Demo 1 kost ~567 input-tokens → **\$0.000024** per beslissing.
Amsterdam-Californië alleen al is ~140 ms RTT: de vloer voor EU-gebruikers ligt hoger dan de vendor-vloer.

Head-to-head · dezelfde beslissingen door het LLM en door Jev

OFFLINE - SYNTHETISCHE PLACEHOLDER, geen echte Jev-output

6.3×

triage: sneller (mediaan)

n/a

triage: goedkoper

94%

triage: zelfde antwoord (team 82%, urgency 100%, refund 100%)

4.6×

gate: sneller (mediaan)

n/a

gate: goedkoper

94%

gate: zelfde antwoord (destructive 100%, production 100%, secrets 83%)

taak	systeem	n	mediaan	p95	tokens in / uit	kost per 1000
triage	jev	11	388 ms	439 ms	1525 / 0	\$0.0058
triage	llm	11	2460 ms	2820 ms	3065 / 2315	n/a
gate	jev	6	445 ms	490 ms	900 / 0	\$0.0063
gate	llm	6	2050 ms	2312 ms	1800 / 1110	n/a

LLM: placeholder:no-llm-was-called met structured output en **dezelfde criteria-tekst** · Jev: jev-placeholder (synthetic, not a real model answer) · 4 calls parallel · kost: Jev \$0.042 / M

input-tokens (vendor), LLM volgens `LLM_PRICE_*` in `.env` · *zelfde
antwoord* is overeenstemming tussen de twee, geen ground truth

Eén call tegenover twintig samples

OFFLINE - SYNTHETISCHE PLACEHOLDER, geen echte Jev-output
T-109 · My subscription renewed but the new features from the upgrade aren't showing. Did the payment go through or is...

team	LLM: aandeel van 20 samples	Jev: kans, 1 call
billing	0.60	0.49
technical	0.35	0.47
sales	0.05	0.02
other	0.00	0.02

60%
LLM eens met zichzelf · zegt zelf 0.88
0.49
Jev confidence · spreiding over 3× 0.00
47 s
20 samples vs 380 ms · n/a vs \$0.000004
T-102 · I was charged EUR 49 twice this month. Please refund one of them.

team	LLM: aandeel van 20 samples	Jev: kans, 1 call
billing	1.00	0.96
technical	0.00	0.02
sales	0.00	0.01
other	0.00	0.01

100%
LLM eens met zichzelf · zegt zelf 0.97

0.96

Jev confidence · spreiding over 3×0.00

47 s

20 samples vs 380 ms · n/a vs \$0.000004

Een LLM-antwoord is **één trekking** uit een verdeling die je nooit ziet; vraag het twintig keer en je reconstrueert ze voor twintig keer de prijs. Jev geeft die verdeling in één call. LLM: `placeholder:no-llm-was-called` · Jev: `jev-placeholder (synthetic, not a real model answer)`

Kanttekeningen

- **Early access:** API en modelnamen kunnen nog bewegen (`jev-latest` is een alias)
- **Vendor-benchmarks:** 193x/444x zijn hun cijfers op hun workflows
- **Latency vanuit de EU:** server aan de US West Coast, reken op de RTT er bovenop
- **Alpha-packages:** `langchain-typesafe` 0.0.1a3, eerste release 17 sept 2026; `typesafe-sdk` 0.7.0
- **Geen reasoning-output:** je krijgt kansen, geen *waarom* — debuggen = state en vragen aanpassen
- **Gecalibreerd $\neq$ correct:** typed output garandeert de interface, niet de waarheid; valideer op je eigen data
- **Confidence** meet concentratie van de verdeling, geen toestemming om te handelen
- Drempels zijn **beleid in code:** evalueer ze op je eigen consequenties

Takeaway

Laat het LLM praten en plannen. Laat System 1 kiezen.

- Beslissingen die nu via *prompt* → *tekst* → *parse* lopen, zijn kandidaten voor **Choice / Score / Noul**
- Kansen + drempel in **jouw** code = een gate die je kan testen, loggen en bijsturen
- Begin met één *smart if* in een bestaande flow; meet latency en kwaliteit op je eigen data

github.com/koenae/jev-demo

alle demo's: `uv run python demos/demo1_smart_if.py --offline`

slides: `uv run marimo run presentation/talk.py`